`switch`, `getchar()`, Strings and Character arrays

Sudeep K S, Mahindra University (October 2026)

A **switch** statement is similar to if-else statement. It is used when we need the program to do different things based on different values of some variable or expression.

Syntax:

```
switch <expression> {
   case <value 1>:
      <statements to execute if expression == value 1>
      break;
   case <value 2>:
      <statements to execute if expression == value 2>
      break;
   case <value 3>:
      …
   default:
      <statements that execute if none of the above cases match>
      break;
}
```

Below is an example:

```
switch (a % 3) {
      case 0:
        printf("a is divisible by 3\n");
        break;
      case 1:
        printf("dividing a by 3 gives remainder 1\n");
        break;
      case 2:
        printf("dividing a by 3 gives remainder 2\n");
        break;
}
```

Note that instead of a single `printf` statement, we can have a sequence of statements (to be executed for the case). In this example, `'case 2` can be replaced with `'default'`, both will give the same result.

The 'default' option is particularly useful when we want to merge several remaining cases into one, as in the below example.

```
switch (a % 5) {
      case 0:
        printf("a is divisible by 5\n");
        b = a;
        break;
      case 1:
        printf("dividing a by 5 gives remainder 1\n");
        b = a + 4;
        break;
      default:
        printf("dividing a by 5 gives a remainder 2, 3 or 4\n");
        b = a * 2;
        break;
}
```

It is important to put the 'break' statements before moving on to the next case. If that is not present, the statements given under remaining cases will also run after completing the current case statements. For example, the code snippet

```
int a = 10;
switch (a % 3) {
      case 0:
        printf("a is divisible by 3\n");
        break;
      case 1:
        printf("dividing a by 3 gives remainder 1\n");
      case 2:
        printf("dividing a by 3 gives remainder 2\n");
        break;
}
```

Prints the following (because there is no break statement before case 2):

```
dividing a by 3 gives remainder 1
dividing a by 3 gives remainder 2
```

Note that whatever can be done with 'switch' can also be done with if-else statements, but switch is a more elegant way of doing it when there are three or more cases to deal with.

getchar() is a function similar to scanf that reads a single character from the input.

Syntax:

```
c = getchar(); // reads one character from input, assigns value to c
```

This can also be done in a loop. For example, the following code reads characters one by one from the input till it sees a '#' character and prints them out one by one.

```
char c;
while ((c = getchar()) != '#') {
   printf("the character read is %c", c\n);
}
```

Exercise: Run the above code with input given below, and see the output.
a bc de1#2 b

Strings (character arrays) in C

A **string** (defined as type char*) is the same as a **character array** (char <array name>[]) in C. It is a sequence of characters, stored in consecutive locations in memory. It can be declared and assigned value in different ways (as seen below) but **all of them are equivalent**.

```
char* name = "Amal";
char name[] = "Amal";
char name[5] = "Amal";
char name[] = {'A', 'm', 'a', 'l', '\0'};
char name[5] = {'A', 'm', 'a', 'l', '\0'};
```

No matter what way (among the above) you choose, these characters get stored as a sequence of characters one after the other in memory, ending with a null character ('\0'). Their values can be accessed and printed in sequence using any one of the following ways.

1. Using a for loop, accessing the characters as elements of an array:
   ```
   for (i = 0; i < 4; i++) {
      printf("name[%d] is %c\n", i, name[i]);
   }
   ```
2. Using a for loop, accessing the characters with pointer dereference:
   ```
   for (i = 0; i < 4; i++) {
      printf("%d th character is %c\n", i, *(name + i));
   }
   ```

3. Using a while loop, accessing the characters as array elements, till you hit a null character:

```
i = 0;
while (name[i] != '\0') {
   printf("name[%d] is %c\n", i, name[i]);
   i++;
}
```

4. Using a while loop, accessing the characters with pointer dereference, till you hit null:

```
i = 0;
while (*(name + i) != '\0') {
   printf("%d th character is %c\n", i, *(name + i));
   i++;
}
```

All the four methods given above work the same way and give exactly the same results.

This gives us an idea about the **pointer** (or **address** or **location**) usage in C. 'name' is the starting location (or starting address) of this sequence of character variables in memory; name + 1 is the next location (next address), name + 2 is the next and so on. Note that 'name' is also the array name, so the array element at index 2, given by name[2], is same as the character variable value stored at the location (name + 2), given by *(name + 2). We will see pointers in more detail in another set of notes.

We end this by giving two functions that take character arrays as their arguments.

1. A function that takes one array as argument and prints all the characters one by one.

As explained earlier, this can be done in multiple ways. Two are given below.

```
void printstring(char a[]) {
    int i = 0;
    while (a[i] != '\0') {
      printf("a[%d] is %c\n", i, a[i]);
      i++;
    }
}

void printstring(char a[]) {
    while (*a != '\0') {
      printf("character is %c\n", *a);
      a++;
    }
}
```

Note that it can also be done using for loop if the string size is passed into the function.

2. Compare two strings (character arrays), return true if they are same.

   One solution is given below. You may try it in different ways.

```
bool equal(char a[], char b[]) {
   while (*a != '\0' || *b != '\0') // go in if at least one string hasn't ended
   {
      if (*a == '\0' || *b == '\0') // one string ended early, strings not equal
         return false;

      if (*a == *b) {
         a++; b++; // move forward in both the strings
      } else {
         return false; // mismatch found
      }
   }
   return true; // both strings ended, no mismatch found
}
```

Note that if you want the function to return boolean type like this, we need to include bool.h, with the statement `#include <stdbool.h>` in the beginning.

The same function can also be defined as `bool equal(char* a, char* b)`, without any change in the body of the function. That function can also be called exactly the same way and will give the same results.